Department of Computer Science

COS284 Practical 4: Deterministic Finite Automata



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Online publication: 16 October 2026

Copyright © 2026 – All rights reserved.

1 Introduction

A deterministic finite automaton (DFA) is read from a file, built in memory, and run on input strings to decide whether it accepts them. You will write two assembly functions, both called from a C program, so they must follow the calling convention. This practical is the capstone: it uses file I/O, parsing, dynamic memory, structs, and control flow together.

The DFA file format

A DFA is given in a file with this format:

```
3,6
0,1,2
0
0,1,a
0,1,b
1,2,a
1,2,b
2,0,a
2,0,b
```

- The first line gives the number of states and the number of transitions, separated by a comma.
- The second line lists the state IDs, comma separated. The states are numbered 0 to `numStates` − 1.
- The third line lists the IDs of the accepting states, comma separated.
- Each remaining line is one transition, in the form `from,to,symbol`.
- The start state is always 0.

The structs

```
typedef struct { int id; bool isAccepting; } State;
typedef struct { int from; int to; char symbol; } Transition;
typedef struct {
    State* states;
```

```
    Transition* transitions;
    int numStates;
    int numTransitions;
    int startState;
} DFA;
```

You are not given these definitions for assembly. Your code must use the same memory layout C uses for these structs.

2 The functions

Task 1: readDfa (18 marks)

```
DFA* readDfa(const char* filename);
```

Read the file, build the DFA in memory, and return a pointer to it. Return `NULL` if the file cannot be opened.

Task 2: simulateDfa (12 marks)

```
bool simulateDfa(DFA* dfa, const char* inputString);
```

Return `true` if the DFA accepts `inputString`, and `false` otherwise. The input is a null-terminated C string.

3 Building and testing

A makefile and a sample harness `main.c` are provided, along with sample DFA files:

```
make                        # builds ./dfa
./dfa dfa1.txt abababb      # reads dfa1.txt, then simulates "abababb"
```

The marker uses different data and its own harness, so a correct run of `./dfa` does not guarantee marks.

4 Submission and grading

Submit `task1.asm` (`readDfa`) and `task2.asm` (`simulateDfa`), each making its function `global`. Each task is marked on its own: the marker links your file with a C harness, runs it, and compares the printed result byte for byte. Do not print anything yourself; return the result.

Plagiarism policy

It is in your own interest that you, at all times, act responsibly and ethically. As with any work done for the purpose of your university degree, remember that the University of Pretoria will not tolerate plagiarism. Do not copy another student's work or allow another student to copy yours. Doing so constitutes plagiarism, and apart from not gaining the intended experience, you may face disciplinary action as a result. See `http://www.library.up.ac.za/plagiarism/index.htm`.

5 Mark distribution

Task	**Mark**
Task 1: readDfa	18
Task 2: simulateDfa	12
Total	**30**